LangChain OSS
@LangChain_OSS

Organizing Context in a Multi-Agent Harness

Most harnesses support a subagents feature to spawn new tasks from a supervisor agent. Subagents enable parallel reasoning and context isolation, allowing a supervisor agent to delegate work without polluting the context window.

Supervisor agents specify the task, and subagents typically complete the task in a fresh context window. This can lead to waste: subagents may redo context-gathering operations, like file reads, already done by the supervisor.

For cases where subagents can benefit from the supervisor agent's context, we built forked subagents. Forked subagents inherit the supervisor's full conversation instead of starting fresh. Forking can be faster and cheaper than isolated subagents, since reusing the supervisor's conversation takes advantage of prompt caching and reduces repeated work.

Harnesses with subagents

Delegating tasks to subagents is one effective way that an agent can manage its own context. Subagents provide context isolation, so that the details of individual tasks can be withheld from a supervisor agent's context window. If you're curious to learn more, we've written at length about different multi-agent architectures!

The supervisor is one of the most generalizable patterns, and most coding harnesses have adopted it. Here, a supervisor maintains a plan and delegates work to specialized subagents. For example,

- Workers: addressing some well-scoped implementation.
- Reviewers: independent judgment on work already done.

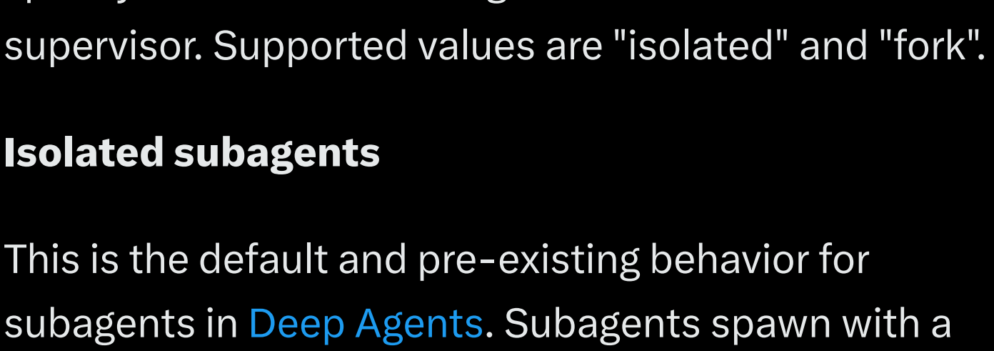
The supervisor agent typically receives just the outcome of a task from the subagents; their intermediate reasoning is withheld from its context window. However, what context subagents should receive from the supervisor is dependent on what the subagent is used for.

Context modes for subagents

To help specify this, we introduced context modes in the latest version of deepagents. Context modes specify what context subagents can receive from the supervisor. Supported values are "isolated" and "fork".

Isolated subagents

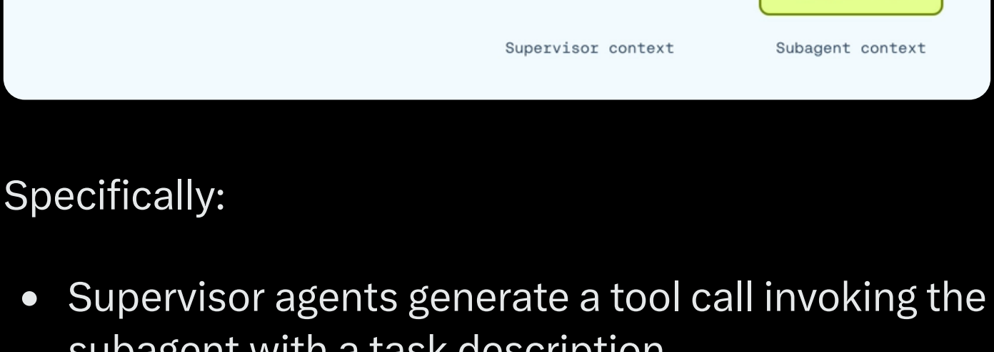
This is the default and pre-existing behavior for subagents in Deep Agents. Subagents spawn with a fresh context window, receiving only the task description specified by the supervisor.



```
{
  "name": "verifier",
  "mode": "isolated",
  "description": "..."
}
```

Forked subagents

Set `"mode": "fork"` and the supervisor's current state propagates to the subagents instead of starting it empty. This is effectively a forked continuation of the current thread— with an added directive written by the supervisor— that is finally unwound into a single tool result read by the supervisor.



```
{
  "name": "worker",
  "mode": "fork",
  "description": "..."
}
```

Specifically:

- Supervisor agents generate a tool call invoking the subagent with a task description.
- The subagent receives the entire supervisor agent's state, including conversation history. The trailing tool call is excised, and its task description is formatted into a user message alongside a fixed preamble clarifying its role.
- When the subagent finishes, the supervisor receives its final message as a response to the originating tool call.

Although forked subagents are seeded with more context than isolated subagents, prompt caching is respected by design. In cases where subagents require detailed context to correctly perform their tasks, forking can save repeated tool calls and context-gathering.

Choosing a context mode

The right choice of context mode depends on the subagent's relationship to the work. A useful way to think about them is with two common patterns: workers that continue the supervisor's work, and verifiers that evaluate it independently.

Worker agent: continue work already in progress.

A worker carries out a piece of work after the supervisor has already gathered context or made a decision. For example, the supervisor might inspect an error, trace it to a particular function, and then delegate the implementation and testing of a fix.

Starting the worker in isolation would force it to rediscover its evidence. With `fork`, it receives the supervisor's history and can pick up where the investigation left off. A supervisor calls this when some work needs to be done, but doesn't necessarily care about the intermediate steps it takes to arrive to a conclusion.

```typescript
const fixerSubagent: SubAgent = {
  mode: "fork",
  name: "fixer",
  description:
    "Use when a problem has already been
  systemPrompt: "...",
}
```

The supervisor might invoke it with a task like:

> Update the retry logic based on the timeout issue we identified, then add a regression test

---

Verifier agent: independently evaluate work

A verifier reviews another agent's work against some criteria- for example, checking a diff for correctness, backwards compatibility, and test coverage.

In this case, inheriting the supervisor's reasoning can be counterproductive. The verifier should evaluate the work itself rather than being anchored by the supervisor's diagnosis or expectations. isolated mode gives it the task and relevant review materials without the preceding conversation.

```typescript
const reviewerSubagent: SubAgent = {
  mode: "isolated",
  name: "reviewer",
  description:
    "Use after an implementation is compl
  systemPrompt: "...",
}
```

The supervisor might invoke it with:

> Review this diff for completeness, backwards compatibility, and adequate test coverage.

We've previously written about RubricMiddleware which is another instance of using an independent verifier!

Specializing subagents

Alongside things like tools and middleware, context modes are one of the levers you can use to specialize subagents to a task. Here's a few subagents we consider specialized and their relationship to context modes:

Researcher agent: investigate a question

A researcher investigates a question and returns a condensed answer to the supervisor. For example, a supervisor might delegate separate questions about an unfamiliar library, a competitor, or the history of a technical decision.

When the question can stand on its own, the researcher does not need the supervisor's conversation. Using isolated keeps its context focused on the question at hand. This is especially useful when several researchers run in parallel: forking each one would duplicate the supervisor's history even though each researcher only needs its assigned question.

```typescript
const researcherSubagent: SubAgent = {
  mode: "isolated",
  name: "researcher",
  description:
    "Use to investigate a self-contained
  tools: [search_engine],
};
```

The supervisor might invoke it with:

> Determine whether the API changed between versions 1.2 and 1.3, and link to the relevant release notes.

We can give the subagent its own capabilities (like a `search_engine` tool) to help it complete its task.

Memory agent: retain information from the conversation

A memory agent identifies information from an interaction that should be available later- for example, a user preference, an architectural decision, or a constraint established during the conversation.

Here, the conversation is the material the agent needs to analyze. With `fork`, the memory agent receives the full interaction and can decide what is worth preserving without requiring the supervisor to restate it in the task.

```typescript
const memorizerSubagent: SubAgent = {
  mode: "fork",
  name: "memorizer",
  description:
    "Use when the conversation contains d
  permissions: [
    {
      operations: ["write"],
      paths: ["/**"],
      mode: "deny",
    },
    {
      operations: ["read"],
      paths: ["/AGENTS.md", "/docs/**"],
      mode: "allow",
    },
  ],
};
```

The supervisor might invoke it with:

> Memorize the decisions and preferences established in this conversation.

Because we want to limit exactly what the memorizer agent can edit, we can specialize the subagent by setting restrictions on what files it can edit while it works.

Try it

`deepagents` is a framework we're building that takes our lessons learned from working with thousands of different teams shipping agents. You can try subagent context modes (see the docs here), and much more by installing:

```bash
# Python
uv add deepagents
# Typescript
pnpm i deepagents
```

Let us know what you think via GitHub issues, the forum, or on X / LinkedIn!

Written by @t_bengre and @chester_curme. This blog is also available to view on https://www.langchain.com/blog/organizing-context-in-a-multi-agent-harness

0:10 · 9 Sep 2026 · **75.9k** Views

3 28 189 347

FastFix AI @FastfixAI · 51m
What's your retry budget
7

EKOS _ AGI 🦊 🇮🇷 @OMID_0909 · 1h
Fork solves context continuity, but not context trust. Passing the supervisor's full history forward is useful only if the evidence inside that context is traceable and verifiable. Otherwise, you're just propagating the same hidden assumptions faster.
10

0xSean @bookingX0 · 13h
supervisor 把 context 隔离做好了，但 subagent 之间怎么共享状态这块没说清楚吧
350